Logistic Regression

A true beginner's guide — everyday examples first, maths only when you're ready

Prepared for Kishore Kunal (38BM21). Assumes zero stats background.

*How to read this: every idea starts with a **real-life scene** (pink box), then plain English, then the proper term. The maths sits in **dashed grey "For the record" boxes you can skip on your first read** — the guide makes full sense without them. Come back for the maths on your second pass, before the exam.*

What's inside

1. Why a straight line can't answer yes/no
2. The S-curve (sigmoid)
3. Odds & log-odds, the easy way
4. Full worked example: age → subscribe
5. Reading the coefficients
6. The threshold: where to draw the line
7. Was the model right? The score-card
8. Accuracy, precision, recall
9. When accuracy lies (the paradox)
10. F1 score — the tie-breaker
11. ROC & AUC — testing every line
12. Picking the best cut-off
13. More than two answers (bonus)
14. Glossary
15. Cheat sheet

1. Why a straight line can't answer yes/no

PICTURE THIS

You run a small bank. A loan officer looks at how much someone earns and decides: *give the loan, or don't.* You want a computer to learn this pattern from past decisions.

You've already met linear regression. You give it an input — say, monthly income — and it predicts a **number**: "this house will sell for ₹82 lakh," or "this student will score 78 marks." The answer can be any number, big or small.

But the loan question isn't a number. The answer is one of exactly two words: **Yes** or **No**. When the answer is a category like this instead of a number, we call it a **classification** problem.

THE TERM

Classification = predicting a *category* (Yes/No, Spam/Not-spam, Fraud/Genuine). **Regression** = predicting a *number*. Logistic regression is our tool for the Yes/No kind.

So why not just reuse the straight line?

Let's try. We label every past "Yes" as 1 and every "No" as 0, and ask the old straight line to fit. It falls apart in two ways — and here's each one with a concrete case:

EXAMPLE — THE LINE GIVES NONSENSE ANSWERS

Say the fitted line is: *score = income ÷ 20,000 − 1*.

- A customer earning ₹80,000 → score = 80,000 ÷ 20,000 − 1 = 4 − 1 = **3**.
  But a Yes/No answer should sit between 0 and 1. What is a "**3** Yes"? Meaningless.
- A customer earning ₹10,000 → score = 0.5 − 1 = **−0.5**.
  A *negative* Yes? Also meaningless.

Problem 1: a straight line shoots off past 1 and below 0, but a yes/no answer must stay inside 0 to 1 (think of it as the *chance* of Yes).

EXAMPLE — ONE WEIRD CUSTOMER TILTS EVERYTHING

Your data forms two flat rows: rejected people (all at 0) and approved people (all at 1). A straight line has to slice diagonally through both rows and fits neither well.

Now one billionaire applies (income ₹50,00,000). To pass through that one far-out point, the line tilts steeply — and suddenly it mis-scores all your ordinary customers.

Problem 2: a single extreme case drags the whole line and wrecks the everyday predictions.

EXAM HOOK

"Why can't linear regression be used for a yes/no outcome?" → Two reasons: **(1)** its answers aren't stuck between 0 and 1, so they aren't valid chances; **(2)** a straight line fits two-flat-rows data badly and gets dragged by extreme points. **The fix is an S-shaped curve, not a straight line.**

2. The S-curve (the sigmoid)

PICTURE THIS

Think about how confident you are that it'll rain, as clouds gather. On a clear morning: basically 0% — no chance. As clouds thicken: your confidence climbs, slowly at first, then *quickly* once it gets grey, then it flattens near 100% once the sky is black. It never goes below 0% or above 100%.

That rising, flattening shape — low and flat, then a quick climb, then high and flat — is exactly what we need. Drawn out, it looks like the letter **S** lying on its side. It starts near 0 ("almost certainly No"), ends near 1 ("almost certainly Yes"), and swings through the uncertain middle. Best of all, it *never* escapes 0 or 1 — so Problem 1 from before is fixed for free.

THE TERM

This S-shaped curve is the **sigmoid function** (also called the **logistic function** — that's where "logistic regression" gets its name). We write it with the Greek letter sigma, σ.

In plain words, the sigmoid is a **squasher**: hand it any number at all — huge, tiny, negative — and it politely hands back a chance between 0 and 1.

EXAMPLE — THE SQUASHER IN ACTION (NO FORMULA NEEDED TO SEE IT)

- Feed it a **big positive** number (like +5) → it returns about **0.99** → "almost certainly Yes."
- Feed it **0** (perfectly on the fence) → it returns exactly **0.5** → "50/50, a coin flip."
- Feed it a **big negative** number (like −5) → it returns about **0.01** → "almost certainly No."

See the pattern? Whatever goes in, what comes out is always a sensible chance. The number 0 is the tipping point that lands bang on 50%.

FOR THE RECORD — THE SIGMOID FORMULA *skip on first read*

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

In plain English: z is the number you feed in. e is just a fixed constant ≈ 2.718 (like π ≈ 3.14). The formula is built so big-z → near 1, small-z → near 0, and $z = 0$ → exactly 0.5. Checking the middle by hand: at $z = 0$, $e^{-0} = 1$, so σ = 1 ÷ (1 + 1) = **0.5**. ✓

EXAM HOOK

Three facts examiners love: the sigmoid **takes in any number (−∞ to +∞)** but **outputs only 0 to 1**; that output is read as a **chance/probability**; and **feeding in 0 gives exactly 0.5**.

What number do we feed the squasher? (The link to linear regression)

Here's the neat bit. The number we feed the sigmoid is *your old friend, the straight line*. We take the linear-regression score — intercept plus slope times input — and pipe it into the squasher. The line does the "raw scoring," the sigmoid turns that raw score into a proper chance.

ONE-LINE SUMMARY

Logistic regression = linear regression's straight line, with a **squasher (sigmoid) bolted on the end** to convert the raw score into a Yes/No chance between 0 and 1.

EXAMPLE — LINE + SQUASHER TOGETHER

Suppose the line gives a customer a raw score of **2**.
Feed 2 into the squasher → out comes about **0.88**.
So we'd say: "88% chance this customer says Yes."
Another customer scores **−1** → squasher → about **0.27** → "27% chance of Yes."

EXAM HOOK — HOW THE LINE IS FOUND

In *linear* regression we fit the line by **making the squared errors as small as possible (OLS)**. In *logistic* regression that trick doesn't work (the curve isn't straight anymore), so we use **gradient descent** — the same downhill-rolling method you already learned — guided by a principle called **Maximum Likelihood Estimation (MLE)**. Just remember the names.

3. Odds and log-odds, the easy way

PICTURE THIS

A cricket bookie says the odds of India winning are **4 to 1**. Your friend says "so that's an 80% chance." Both are describing the *same* belief — just in two different languages. Learning to translate between them is this whole section.

Chance vs Odds — two ways to say the same thing

Chance (probability) answers: *out of everything, what share is Yes?* It runs from 0 to 1 (0% to 100%).

Odds answers a slightly different question: *how many Yeses for each No?* "4 to 1" means four wins for every one loss.

EXAMPLE — TRANSLATING BOTH DIRECTIONS

Chance → Odds. Suppose the chance of rain is 0.8 (80%). Then the chance of *no* rain is 0.2. So for every 0.2 of "no rain" there's 0.8 of "rain" → 0.8 ÷ 0.2 = **4**. The odds are **4 to 1**. (Matches the bookie exactly.)

Odds → Chance. Someone tells you the odds are 4. To get the chance back: 4 out of (4 + 1) total parts = 4 ÷ 5 = **0.8** → 80%. Back where we started. ✓

FOR THE RECORD — THE TWO LITTLE FORMULAS *skip on first read*

$$\text{Odds} = \frac{P}{1 - P} \qquad P = \frac{\text{Odds}}{1 + \text{Odds}}$$

In plain English: P is the chance. "Odds = chance of Yes ÷ chance of No." And you can always flip back the other way.

EXAM HOOK

"4 to 1 odds" is **NOT** a chance of 4 — it's a chance of **80%**. Odds and chance are different numbers for the same belief. Know both translations.

One more step: log-odds (why we bother)

Remember the real goal: we want to fit a *straight line*. But a straight line can output any number from minus-a-lot to plus-a-lot, while our *chance* is trapped between 0 and 1. Mismatched ranges. So we stretch the chance out until it can match a straight line's range. Watch the range grow in three steps:

Chance

0 → 1 ▶ **Odds**

0 → huge ▶ **Log-odds**

−huge → +huge

Taking the **logarithm of the odds** is the final stretch. It opens the range all the way out to "any number at all" — which is *exactly* what a straight line can produce. This stretched quantity is called the **log-odds**, or the **logit**.

THE WHOLE TRICK, IN ONE BREATH

We can't draw a straight line through the *chance* (it's boxed in 0–1). So we translate: Yes/No → chance → odds → **log-odds** (which isn't boxed in). We fit our straight line to the log-odds, then translate all the way back to a chance at the end. That round-trip *is* logistic regression.

EXAMPLE — FEEL THE STRETCH WITH NUMBERS

A near-certain "No" has chance 0.01 → odds ≈ 0.01 → log-odds ≈ **−4.6** (way down low).
A coin-flip has chance 0.5 → odds = 1 → log-odds = **0** (dead centre).
A near-certain "Yes" has chance 0.99 → odds ≈ 99 → log-odds ≈ **+4.6** (way up high).

See how the cramped 0-to-1 chances got spread out from −4.6 to +4.6? Now a straight line can comfortably reach every value.

FOR THE RECORD — THE LOGIT *skip on first read*

$$y^* = \ln\frac{P}{1-P} = b_0 + b_1 x$$

In plain English: "ln" means natural logarithm — the stretch. The left side is the log-odds (we nickname it *y**). The right side is literally the straight line. So: *log-odds = a straight line.* Everything you know about lines — intercept, slope — applies here.

EXAM HOOK

"What does the logit / log-odds transformation achieve?" → It **stretches a boxed-in 0-to-1 chance into an open −∞ to +∞ range** so a straight line can fit it. Line phrase: "chance ranges 0 to 1, but log-odds ranges −∞ to +∞."

4. Full worked example — will this customer subscribe?

THE TASK

A magazine wants to guess whether someone will subscribe, based only on their **age**. Older readers tend to subscribe more. The computer has already learned the pattern from past customers — our job is to *use* it on a new 35-year-old.

The learned rule (given to us) is: *log-odds = −26.524 + 0.781 × age*. In plain terms, that's the straight line, with a starting value of −26.524 and a climb of 0.781 for each year of age. Let's walk one person through, one tiny step at a time.

WORKED — A 35-YEAR-OLD, STEP BY STEP

Step 1 — get the raw score (the straight line).
Plug age 35 in: −26.524 + 0.781 × 35 = −26.524 + 27.335 = **0.813**.
Plain English: this is the "log-odds" — just a raw score for now, not yet a chance.

Step 2 — squash it into a chance (the sigmoid).
Feed 0.813 into the squasher. Out comes ≈ **0.69**.
Plain English: a 35-year-old has about a **69% chance** of subscribing.

Step 3 — make the call (the threshold).
Our rule: if the chance is at least 50%, predict Yes. 69% is above 50%, so → **"Yes, will subscribe."**

Now the punchline that trips people up. Let's do the *very next year* and watch what one extra year of age does.

WORKED — A 36-YEAR-OLD, THEN COMPARE

Raw score: −26.524 + 0.781 × 36 = **1.594**. Squash it → ≈ **0.83** → 83% chance.

So going from age 35 to 36 moved the chance from 69% up to 83% — a jump of about **14 percentage points** from just *one* extra year. Hold that thought; the next section explains why that jump isn't always 14 points.

FOR THE RECORD — THE EXACT ARITHMETIC OF STEP 2 *skip on first read*

Squashing 0.813: $p = e^{0.813} \div (e^{0.813} + 1)$. Since $e^{0.813} \approx 2.255$, $p = 2.255 \div 3.255 \approx$ **0.693**. For age 36: $e^{1.594} \approx 4.92$, $p = 4.92 \div 5.92 \approx$ **0.831**. Change = 0.831 − 0.693 = **0.138**.

5. Reading the coefficients (what the numbers mean)

PICTURE THIS

Imagine pushing a child on a swing. When the swing is at the bottom (moving fast), one push sends them much higher. When they're already near the very top, the same push barely lifts them. *Same push, different effect, depending on where they already are.* A logistic coefficient behaves exactly like that push.

In a straight line, the slope is a fixed promise: "each extra year adds the same amount, always." Beginners assume logistic works the same way — but it has a **twist**. The coefficient's effect on the *final chance* depends on where you already are on the S-curve.

THE TWIST TO REMEMBER

The coefficient (here 0.781) adds a **fixed amount to the raw score** every year — that part is constant, like a straight line. But its effect on the **final chance** is *biggest in the steep middle of the S* and *tiny at the flat top or bottom.*

EXAMPLE — THE SWING EFFECT IN NUMBERS

The *same* one-year jump, at two different ages:

Age jump	Chance moves	Where on the S-curve
35 → 36	**+14 points** (69% → 83%)	Steep middle — big push
45 → 46	**+0.01 points** (≈100% → ≈100%)	Flat top — barely moves

Same coefficient, same one extra year — huge effect in the middle, almost nothing at the top. That's the swing.

Two easy reading rules

Rule 1 — the sign. A **positive** coefficient means "more of this pushes the chance *up*." A **negative** one pushes it *down*.

EXAMPLE — A NEGATIVE COEFFICIENT

Add gender to the magazine model and suppose the coefficient for "woman" is **−0.56** (negative). Plain English: being a woman *lowers* the chance of subscribing, when comparing people of the same age. If it had been +0.56, it would *raise* the chance.

Rule 2 — the odds ratio. There's a standard way to report "how strong" a feature is: raise *e* to the power of the coefficient. For age, that's about **2.18**, meaning *each extra year roughly doubles the odds* of subscribing.

FOR THE RECORD — ODDS RATIO *skip on first read*

Odds ratio = $e^{\text{coefficient}}$. For age: $e^{0.781} \approx 2.18$. Reading it: >1 means the feature increases the odds; <1 means it decreases them; exactly 1 means no effect. Careful — the raw coefficient *adds* to the log-odds, while $e^{\text{coefficient}}$ *multiplies* the odds. Different actions.

EXAM HOOK

Two things get tested: **(1)** a logistic coefficient is constant on the *log-odds* scale but *not* on the *chance* scale (the swing/S-curve effect); **(2) $e^{\text{coefficient}}$ = odds ratio** — above 1 helps, below 1 hurts.

Using several inputs at once

Real models use more than one input — say income *and* balance *and* age together. Nothing new happens: the raw-score line just gets more terms (a slope for each input), then the same squasher turns it into a chance. This is called **multiple (or multivariable) logistic regression**. You can even mix number-inputs (age) with Yes/No-inputs (woman = 1 or 0) — those Yes/No inputs are called **dummy variables**.

Health-checks on the model (just know the names)

Check	What it tells you, plainly
Wald's test	"Does this *one* input actually matter?" (per-feature significance)
Likelihood Ratio test	"Does the *whole* model beat a do-nothing guess?" Low p-value (<0.05) = yes, it's worth it.
Pseudo R^2	A rough "how good is the fit" score (the logistic cousin of R^2).
AIC / BIC	Compare two models: reward good fit, *punish* needless complexity. **Lower = better.** BIC punishes complexity harder.

6. The threshold — where do we draw the line?

PICTURE THIS

You wake up and your weather app says "70% chance of rain." Do you carry an umbrella? You've secretly set a rule in your head: "if it's more than 50% likely, I'll take it." That personal cut-off is a **threshold** — and a different person might set theirs at 30% (carries it just in case) or 80% (hates carrying it).

The sigmoid hands us a *chance* (like 70%). But the boss wants a decision: Yes or No. So we pick a cut-off. By default we use **0.5 (50%)**: at or above → say Yes; below → say No.

TWO TERMS, DON'T MIX THEM UP

Threshold = the cut-off on the *chance* (default 50%). **Decision boundary** = the matching point on the *input* side where your answer flips from No to Yes. Move one and the other moves too.

EXAMPLE — PROMOTION DECISION (FROM THE COURSE)

An employee gets promoted if their performance rating is above **3.2**. At exactly 3.2, the model's chance of promotion works out to **65%**.

- The rating **3.2** is the **decision boundary** (the input value where the call flips).
- The **65%** is the **threshold** (the chance at that flip point).

Rate above 3.2 → predicted Yes; below → predicted No.

Crucially, 50% isn't sacred — it's a *business choice*. Screening for a deadly disease? Lower the threshold (say 20%) so you catch more real cases, even at the cost of more false alarms. Approving a big risky loan? Raise it (say 80%) so you only say Yes when very sure. Sections 11–12 show how to choose it properly.

7. Was the model right? The score-card

PICTURE THIS

A doctor orders a cancer test for 100 patients. Afterwards we can sort every result into one of four buckets: the test caught a real cancer, the test correctly said "all clear," the test raised a false alarm, or the test *missed* a real cancer. Those four buckets are the whole game.

To judge any Yes/No model, we lay its guesses against the truth in a little 2×2 grid called the **confusion matrix**. Here it is for the cancer test (Positive = "has cancer"):

		THE TRUTH	
		Really has cancer	Really healthy
MODEL SAID	"Cancer"	TP caught it ✓	FP false alarm
	"Healthy"	FN missed it ✗	TN all-clear ✓

Reading the four buckets in plain English:

- **TP (True Positive)** — said cancer, and there *was* cancer. A correct catch. ✓
- **TN (True Negative)** — said healthy, and they *were* healthy. Correct all-clear. ✓
- **FP (False Positive)** — said cancer, but they were fine. A *false alarm*. Scary letter, needless worry.
- **FN (False Negative)** — said healthy, but they had cancer. A *dangerous miss*. The worst kind here.

EXAM HOOK — THE CLASSIC SWAP

False Positive = Type I error (a false alarm — you "cried wolf"). **False Negative = Type II error** (a real miss — the wolf slipped past). These two get swapped on nearly every exam. Memory aid: Type **1** = you raised **1** false alarm; Type **2** = you were doubly careless and let the real one through.

WHICH MISTAKE IS WORSE? IT DEPENDS

Courtroom (Positive = "guilty, convict them"): a *False Positive* jails an innocent person — unforgivable. So we bend over backwards to avoid FPs.
Cancer or fraud: a *False Negative* lets a real case slip through — possibly fatal or very costly. So we bend over backwards to avoid FNs.
Tuning the threshold (Section 6) is exactly how we choose which mistake to avoid.

8. Accuracy, precision, recall

From those four buckets we build three famous scores. Forget the formulas for a second — here's the everyday question each one is really asking, using the cancer test.

Score	The real question it answers
Accuracy	"Out of *all* patients, what share did the test get right?"
Precision	"When the test *shouts 'cancer!'*, how often is it actually right?" (trustworthiness of a Yes)
Recall	"Out of all the people who *really had* cancer, how many did the test catch?" (how few we miss)

EXAMPLE — ONE SMALL TEST, ALL THREE SCORES

Test 100 people. Results: caught 30 real cancers (TP=30), correctly cleared 55 healthy (TN=55), 10 false alarms (FP=10), and missed 5 real cancers (FN=5).

Accuracy = everything correct ÷ everyone = (30 + 55) ÷ 100 = **85%**.
Plain: the test was right about 85 of the 100 people.

Precision = correct "cancer" shouts ÷ all "cancer" shouts = 30 ÷ (30 + 10) = 30 ÷ 40 = **75%**.
Plain: when it yelled "cancer," it was right 3 times out of 4.

Recall = cancers caught ÷ cancers that existed = 30 ÷ (30 + 5) = 30 ÷ 35 = **86%**.
Plain: of the 35 people who truly had cancer, it caught 30 of them.

FOR THE RECORD — THE FORMULAS *skip on first read*

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} \qquad \text{Precision} = \frac{TP}{TP + FP} \qquad \text{Recall} = \frac{TP}{TP + FN}$$

One more you may see: **Specificity** = TN ÷ (TN + FP) — "of all the truly healthy people, how many did we correctly clear?" It's Recall's mirror image for the No class.

A TRICK TO NEVER CONFUSE PRECISION & RECALL

Precision is about the model's *Yes-claims*: "when it says Yes, can I trust it?" **Recall** is about the *real Yeses*: "did it find them all?" Precision guards against false alarms; Recall guards against misses.

EXAMPLE — WHICH ONE DO YOU CARE ABOUT?

Spam filter → care about **Precision**. A false alarm here means a real, important email gets dumped in the spam folder and you miss it. You'd rather let a little spam through than lose a real email — so be *very sure* before flagging. High precision = few good emails wrongly binned.

Cancer screening → care about **Recall**. A miss here could cost a life. You'd happily accept some false alarms (extra scans) to make sure you catch every real case. High recall = few real cancers missed.

EXAM HOOK

Pick **Precision** when a false alarm is costly (spam). Pick **Recall** when a miss is intolerable (disease, fraud). Also memorise the synonyms: **Recall = Sensitivity = True Positive Rate**, and **False Positive Rate = 1 − Specificity**. You'll need these in the ROC section.

9. When accuracy lies — the accuracy paradox

PICTURE THIS

A lazy weather forecaster in a desert always says "no rain, ever." Since it rarely rains there, he's right about 95% of the time — a fabulous-looking accuracy! But he has never once predicted the actual rainy day. His 95% is *useless*. This is the accuracy paradox, and it's the single most tested idea in this whole topic.

The problem shows up whenever one answer is **rare** (few cancers, little fraud, rare rain). Let's make it concrete with cancer. Out of 300 test patients, only 30 truly have cancer — cancer is the rare class.

The lazy model: "everyone's healthy"

	Really cancer	Really healthy
Said "cancer"	TP = 0	FP = 0
Said "healthy"	FN = 30	TN = 270

WORKED — THE TRAP SPRINGS

Accuracy = (0 + 270) ÷ 300 = **90%!** Looks brilliant.
Precision = 0 ÷ (0 + 0) = **0%** (it never correctly flagged a cancer).
Recall = 0 ÷ (0 + 30) = **0%** (it caught none of the 30 real cancers).

Plain English: the model missed **every single cancer**, yet scores 90% accuracy — purely because healthy people are so common. Deadly and useless, but "90% accurate."

The honest model: actually tries

Now a model that really works — it catches 20 of the 30 cancers, at the cost of 70 false alarms:

	Really cancer	Really healthy
Said "cancer"	TP = 20	FP = 70
Said "healthy"	FN = 10	TN = 200

WORKED — LOWER ACCURACY, BETTER MODEL

Accuracy = (20 + 200) ÷ 300 = **73%** (lower than the lazy model's 90%!).
Precision = 20 ÷ (20 + 70) = **22%**.
Recall = 20 ÷ (20 + 10) = **67%** (it now catches two-thirds of real cancers).

Plain English: this model scores *worse* on accuracy but is *far* more useful — it actually finds cancers.

EXAM HOOK — STATE IT EXACTLY LIKE THIS

The Accuracy Paradox: a model with lower accuracy (73%) can be genuinely better — higher precision AND recall — than a model with higher accuracy (90%). Lesson: on **rare/imbalanced** data, never trust accuracy alone; always check precision and recall too. This is *the* takeaway of the evaluation half.

10. F1 score — the tie-breaker

PICTURE THIS

A job needs someone who is *both* a strong writer *and* a strong coder. A candidate who's brilliant at writing but hopeless at coding shouldn't score well overall — being great at one can't rescue being terrible at the other. We want a combined score that stays low if *either* skill is weak. F1 does that for precision and recall.

Precision and recall often pull against each other. The **F1 score** blends them into one number — but cleverly. It's built so that if *either* precision *or* recall is poor, F1 drops. A model can't hide a terrible recall behind a great precision.

EXAMPLE — WHY A PLAIN AVERAGE WOULD FOOL YOU (BUT F1 WON'T)

A dog-photo detector faces 1,000 photos, only 5 of which are dogs (very rare). It finds 1 dog, calls everything else "not a dog." So it missed 4 dogs but raised no false alarms.

- **Precision** = 1 ÷ 1 = **100%** (perfect — it never wrongly called something a dog).
- **Recall** = 1 ÷ 5 = **20%** (awful — it missed 4 of the 5 dogs).
- **Accuracy** = 996 ÷ 1000 = **99.6%** (looks amazing — the accuracy paradox again!).

A *plain average* of precision and recall = (100 + 20) ÷ 2 = 60% — too flattering. But **F1 = 33%**, correctly telling us the model is poor. F1 refused to let the perfect precision paper over the terrible recall.

FOR THE RECORD — THE F1 FORMULA *skip on first read*

$$F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

In plain English: this is called the **harmonic mean**. Unlike a normal average, it leans toward the *smaller* of the two numbers — so a weak precision or a weak recall drags the whole score down. That's exactly why we use it. Check: 2 × (1.0 × 0.2) ÷ (1.0 + 0.2) = 0.4 ÷ 1.2 = **0.33**. ✓

EXAM HOOK

F1 = harmonic mean of precision and recall (ranges 0 to 1). Use it for **rare/imbalanced data** where accuracy misleads and you need *both* precision and recall to be decent. Rough guide: **F1 ≥ 0.7 is usually "good."** Why harmonic and not plain? Because it *punishes* the weaker of the two, so nothing hides.

11. ROC & AUC — trying every possible cut-off

PICTURE THIS

Back to the umbrella rule. Instead of arguing about whether 50% or 30% is the right cut-off, imagine testing *every* cut-off from 0% to 100% and plotting how well each one does — good catches versus false alarms. That full picture is the **ROC curve**, and the area underneath it (the **AUC**) becomes a single grade for the whole model.

Every score so far assumed one fixed threshold (50%). But we saw the threshold is a dial we can turn. Turn it and you get a different confusion matrix each time. The **ROC curve** shows the model's performance across *all* dial settings at once.

The two things it plots

- **Going up (Y-axis): the good catches** — the share of real Yeses we correctly caught. (Its posh name is True Positive Rate, a.k.a. Sensitivity or Recall.)
- **Going right (X-axis): the false alarms** — the share of real Nos we wrongly flagged as Yes. (Posh name: False Positive Rate.)

In plain terms: **up = good, right = bad.** We want lots of catches without many false alarms — so we want the curve to climb steeply up the left side, hugging the top-left corner.

READING THE SHAPE AT A GLANCE

A great model → the curve leaps up to the top-left corner (catches almost everything, hardly any false alarms).
A useless, coin-flip model → the curve is just the diagonal 45° line (no skill at all).
The closer the curve hugs the top-left, the better. *The area underneath it* measures exactly that.

AUC — turning the curve into one grade

AUC stands for "Area Under the Curve." It squeezes the whole ROC picture into a single number from 0 to 1. Bigger = better. Here's the grading scale examiners use:

AUC	Verdict (plain English)
≈ 0.5	Useless — no better than flipping a coin
0.6 – 0.7	Mediocre
0.7 – 0.85	Very good
0.85+	Excellent
0.999	Too good to be true — *something's wrong* (likely a data leak)

EXAMPLE — COMPARING TWO MODELS WITH ONE NUMBER

Model A has AUC 0.82, Model B has AUC 0.68. Without fussing over thresholds, you can say: **Model A is the better ranker** — across all cut-offs it separates Yeses from Nos more cleanly. AUC lets you compare two models with a single glance.

EXAM HOOK

AUC ≈ 0.5 = random; higher = better. ROC plots *catches* (True Positive Rate, up) against *false alarms* (False Positive Rate, right). A suspiciously perfect 0.999 is a **red flag** (data leakage), not a triumph. The bands (0.7–0.85 = very good) are often asked word-for-word.

WHEN EVEN AUC CAN MISLEAD

If the Yes class is *extremely* rare, AUC can still look rosy while the model quietly misses most real Yeses. In that case, lean on the **precision-recall view and F1** instead. Rule of thumb: extreme imbalance → trust precision, recall, F1 over AUC.

12. Picking the best cut-off

PICTURE THIS

The default 50% umbrella rule is rarely the smartest. If getting caught in rain ruins an important meeting, you'd lower your cut-off to grab the umbrella more often. Choosing the cut-off *on purpose* — instead of blindly using 50% — is what these two methods do.

Method 1 — Youden's Index (the "best all-rounder" point)

This method rewards a cut-off that catches lots of real Yeses *and* correctly clears lots of real Nos — both at once. You try every cut-off and keep the one that scores highest on this balance.

EXAMPLE — PICKING WITH YOUDEN

Suppose at cut-off 0.4 the model catches 85% of Yeses (Sensitivity) and correctly clears 70% of Nos (Specificity). Its Youden score = 0.85 + 0.70 − 1 = **0.55**. You compute this at every cut-off and pick the one with the highest score — that's your best all-round threshold.

FOR THE RECORD *skip on first read*

Youden's J = Sensitivity + Specificity − 1. Pick the cut-off that maximises J. J = 0 means useless (on the diagonal); J = 1 means perfect. Visually, it's the point on the ROC curve that sits highest above the diagonal.

Method 2 — Cost-based (the "cheapest mistakes" point)

Sometimes one mistake genuinely costs more money (or lives) than the other. So we put a price tag on each type of error and pick the cut-off that makes the *total damage* smallest.

EXAMPLE — COSTING THE ERRORS

Missing a cancer (a False Negative) is far worse than a false alarm (a False Positive). Say we price a missed cancer at ₹100 and a false alarm at ₹5. We then pick the cut-off that minimises (₹5 × number of false alarms) + (₹100 × number of misses). Because misses are so expensive, this pushes the cut-off *lower*, so the model catches more real cancers.

FOR THE RECORD

skip on first read

Minimise **[C1 × FP + C2 × FN]**, where C1 = cost of a false alarm and C2 = cost of a miss. Build this asymmetry into training and you get **Cost-Sensitive Logistic Regression (CS-LR)** — in the course table it trades a sliver of accuracy for clearly better recall and F1, exactly the deal you want when misses are expensive.

13. Bonus — when there are more than two answers

PICTURE THIS

Instead of Yes/No, a model sorts flowers into *three* types: Setosa, Versicolor, Virginica. How do we score that? Simple trick: judge one type at a time, treating it as "this type vs everything else," then repeat.

For any one class — say **Setosa** — you re-use the exact same four buckets, just from Setosa's point of view:

- **TP** = times the model said Setosa and it really was Setosa.
- **FN** = times it really was Setosa but the model said something else (the rest of Setosa's row).
- **FP** = times the model said Setosa but it was really another flower (the rest of Setosa's column).
- **TN** = everything that had nothing to do with Setosa either way.

EXAMPLE — SETOSA'S SCORE (COURSE NUMBERS)

The model got 16 Setosas right → TP = 16. It never mislabelled a real Setosa → FN = 0. It never wrongly called something Setosa → FP = 0. Everything else (17 + 1 + 0 + 11) → TN = **29**. So Setosa was classified perfectly.

To get one overall precision/recall across all three flowers, you add up every class's TPs, FPs and FNs first, then apply the usual formulas — this all-in-one approach is called **micro-averaging**. (Just know the name for the exam.)

Glossary — every term in one line

Term	Plain-English meaning
Classification	Predicting a category (Yes/No), not a number.
Sigmoid / logistic function	The S-curve "squasher" that turns any number into a chance from 0 to 1.
Raw score (z)	The straight-line value (intercept + slopes) we feed into the squasher.
Chance / probability (P)	How likely Yes is; from 0 to 1.
Odds	Yeses per No, e.g. 4-to-1. From 0 upward.
Log-odds / logit (y*)	The stretched-out chance that a straight line can actually fit.
Odds ratio	e^(coefficient); how much the odds multiply per one-unit increase in a feature.
MLE / gradient descent	The method used to find the line's numbers (instead of OLS).
Threshold	The chance cut-off for saying Yes (default 0.5).
Decision boundary	The input value where the Yes/No answer flips.
Confusion matrix	The 2×2 score-card: TP, FP, FN, TN.
Type I error	False Positive — a false alarm.
Type II error	False Negative — a missed real case.
Accuracy	Share of all cases got right.
Precision	When it says Yes, how often it's right.
Recall / sensitivity / TPR	Of all real Yeses, how many it caught.
Specificity	Of all real Nos, how many it cleared. (FPR = 1 − specificity.)
F1 score	Blend of precision & recall that punishes the weaker one; best for rare data.
Accuracy paradox	A lower-accuracy model can be better on rare data.
ROC curve	Catches (up) vs false alarms (right) across every cut-off.
AUC	Area under ROC; one grade for the model. 0.5 = random, higher = better.
Youden's Index	Sensitivity + Specificity − 1; maximise it to pick a balanced cut-off.
Cost-based cut-off	Price each error, pick the cut-off with the least total cost.
Wald / LR test	Is one input / the whole model doing real work?
AIC / BIC	Model-comparison scores; lower is better; they punish needless complexity.

One-page cheat sheet

THE WHOLE TOPIC IN ONE PARAGRAPH

A straight line can't do Yes/No (its answers escape 0–1 and get dragged by outliers), so we bolt a **squasher (sigmoid)** onto the line to turn its raw score into a chance from 0 to 1. Behind the scenes the line is actually fitted to the **log-odds** (a stretched version of the chance that a line *can* reach), then translated back to a chance, then compared to a **threshold** (default 0.5) to make the call. We grade the result with a **confusion matrix**, remembering that on rare data **accuracy lies** — so we lean on **precision, recall, F1 and AUC.**

The everyday translations to memorise

- **Sigmoid** = squasher: any number in → a chance (0 to 1) out; feeding in 0 gives exactly 0.5.
- **Odds** "4 to 1" = a **chance of 80%**, not 4. (Odds = chance-of-Yes ÷ chance-of-No.)
- **Log-odds** = the stretch that lets a straight line fit a chance.
- **Positive coefficient** pushes the chance up; **negative** pushes it down.
- **Coefficient effect** is fixed on the log-odds, but on the *chance* it's big in the S-curve's middle, tiny at the ends (the swing).
- **Precision** = trust in a Yes (guards false alarms). **Recall** = catching all the real Yeses (guards misses).
- **False Positive = Type I** (false alarm). **False Negative = Type II** (miss).
- **Accuracy paradox**: 90% accuracy can be worthless on rare data — always check precision & recall.
- **F1** = tie-breaker that punishes the weaker of precision/recall; use on rare data.
- **AUC ≈ 0.5 = random**; higher = better; 0.999 = suspicious, not a win.
- Pick **Recall** when a miss is deadly (cancer); **Precision** when a false alarm is costly (spam).
- Lower **AIC/BIC** = better model.

Formulas (only if you want them)

Name	Formula
Sigmoid (squasher)	$\sigma(z) = 1 \div (1 + e^{-z})$; $\sigma(0) = 0.5$
Odds ↔ Chance	Odds = P ÷ (1−P); P = Odds ÷ (1+Odds)
Log-odds (logit)	$y^* = \ln(P \div (1-P)) = b_0 + b_1 x$
Odds ratio	$e^{\text{coefficient}}$
Accuracy	(TP + TN) ÷ (TP + TN + FP + FN)
Precision	TP ÷ (TP + FP)
Recall / Sensitivity / TPR	TP ÷ (TP + FN)
Specificity	TN ÷ (TN + FP); FPR = 1 − Specificity
F1	2 × (P × R) ÷ (P + R)
Youden's J	Sensitivity + Specificity − 1
Cost cut-off	minimise [C1 × FP + C2 × FN]

Linear vs logistic — at a glance

	Linear regression	Logistic regression
Answers	A number	A chance → a Yes/No
Output range	Any number	0 to 1
Shape	Straight line	S-curve (squasher)
Fitted by	Smallest squared errors (OLS)	Maximum likelihood (gradient descent)
What's straight	The answer itself	The log-odds of the answer
Graded by	R^2, RMSE, residuals	Confusion matrix, precision/recall, F1, AUC

End of beginner's study guide · Logistic Regression · Machine Learning for Managers